



Relatório T1 - Servidor HTTP/1.1 sobre sockets TCP

INTEGRANTES:

Carolina Silveira de Oliveira da Silva

Larissa Oliveira da Silva

Maria Eduarda Schüler

1. PROBLEMA

Neste trabalho construímos, do zero, um servidor web (HTTP/1.1) em Python, usando só a comunicação básica por sockets TCP, sem nenhuma biblioteca que já faça HTTP. Depois testamos o servidor a partir de outros computadores da rede. A pergunta que guia o trabalho é: como o jeito de funcionar do TCP afeta a velocidade que a gente percebe no HTTP?

Toda conexão TCP nova começa com uma "apresentação" entre os dois lados, o handshake, que leva pelo menos um RTT (o tempo de ida e volta de um pacote) antes de qualquer pedido sair. As conexões persistentes do HTTP/1.1 servem para pagar esse custo uma vez só, e ele só aparece de verdade com um RTT real entre as máquinas, por isso nada foi medido no mesmo computador. A Parte 1 é o servidor (GET, HEAD, códigos de resposta, segurança e vários clientes ao mesmo tempo). A Parte 2 acrescenta as conexões persistentes e compara usar uma conexão por pedido (C1) com usar uma conexão para todos os pedidos (C2).

Os testes foram feitos em duas rodadas, em redes diferentes, com dois computadores Windows 11: o A (servidor) com Python 3.10.11 e o B (cliente) com Python 3.14.5. O servidor usa a porta 8080 e aceita conexões de qualquer computador da rede. Como clientes usamos o curl, o navegador Chrome e programas curtos em PowerShell.

Rodada	Rede	IP do A (servidor)	IP do B (cliente)
1 (01/10/2026)	192.168.0.x	192.168.0.11	192.168.0.3
2 (04/10/2026)	192.168.1.x	192.168.1.12	192.168.1.13

2. MODELAGEM DO PROBLEMA

Como o servidor é organizado

Tudo está em um arquivo, o server.py, que é iniciado com o comando `python server.py --port 8080 --root ./www` (porta e pasta que será servida). O programa fica esperando clientes e, para cada um que chega, cria uma thread (uma linha de execução) só para ele. Cada parte tem um papel:

Parte (função)	O que faz
Esperar clientes (main)	Abre a porta e cria uma thread para cada cliente que se conecta.
Atender um cliente (atender)	Recebe os bytes e vai juntando até achar o fim do cabeçalho (uma linha em branco). Trata o pedido, responde e decide se mantém a conexão aberta.
Entender o pedido (interpretar_requisicao)	Confere o método, o caminho, a versão e os cabeçalhos (no HTTP/1.1 o Host é obrigatório). Se algo estiver errado, responde 400.
Achar o arquivo (resolver_arquivo)	Troca códigos como %20 pelo caractere, junta o caminho com a pasta raiz e confere se o arquivo está mesmo dentro dela.
Montar a resposta (responder e outras)	Escolhe 200, 403, 404 ou 405 e envia o cabeçalho (com Date, Server, Content-Type e Content-Length) e o arquivo. No HEAD, só o cabeçalho.

Vários clientes ao mesmo tempo

Escolhemos uma thread por conexão. O servidor passa quase todo o tempo esperando dados da rede, então uma thread parada não atrapalha as outras: um cliente lento, ou que abre a conexão e não manda nada, só prende a própria thread, e o servidor continua aceitando clientes novos. A alternativa seria um único laço vigiando todas as conexões, que aguenta mais clientes, mas é bem mais difícil de programar e de explicar. Para poucos clientes, como no laboratório, a thread por conexão é mais simples. O ponto fraco é que cada thread gasta memória.

Segurança e conexões persistentes

Para não entregar arquivos de fora da pasta raiz, o servidor primeiro troca códigos como %2e pelo caractere normal, depois calcula o caminho final do arquivo e só entrega se ele ainda estiver dentro da raiz (senão, 403). Assim, tentativas com `../` e com `%2e%2e` dão o mesmo resultado.

O TCP entrega um fluxo de bytes, sem separar os pedidos. Por isso o servidor guarda tudo o que chega e só começa a tratar o pedido quando encontra o fim do cabeçalho, deixando o resto para o próximo. Na

Parte 2, o servidor mantém a conexão aberta depois de responder, a menos que o cliente peça Connection: close, e fecha sozinho depois de 5 segundos sem atividade. Cada conexão também liga a opção TCP_NODELAY, que manda os dados na hora, sem esperar confirmação. O motivo está nos resultados de C1 e C2.

3. SOLUÇÃO E EXEMPLOS

Os pedidos abaixo foram feitos do computador B para o servidor do A (rodada 1: 192.168.0.11; rodada 2: 192.168.1.12). Os prints originais ficam na pasta Anexos.

Códigos de resposta

Código	Pedido (comando)	Resposta obtida
200 OK	curl.exe -i http://192.168.0.11:8080/a.txt	200 OK, tipo text/plain, tamanho 14, corpo "arquivo a".
200 (HEAD)	curl.exe -I http://192.168.0.11:8080/a.txt	200 OK com os mesmos cabeçalhos do GET e sem corpo.
400 Bad Request	Socket em PowerShell: "LIXO" e "GET / HTTP/1.1" (sem Host)	400 Bad Request nos dois casos.
403 Forbidden	curl.exe --path-as-is -i "http://192.168.0.11:8080/./server.py"	403 Forbidden.
404 Not Found	curl.exe -i http://192.168.0.11:8080/nao-existe	404 Not Found.
405 Method Not Allowed	curl.exe -i -X POST -d "x" http://192.168.0.11:8080/	405 com o cabeçalho Allow: GET, HEAD.

O 400 foi gerado com socket porque o curl corrige a primeira linha e não deixa enviar um pedido inválido.

Tentativas de sair da pasta raiz

#	Pedido enviado	Resposta	Rodada
1	GET ../server.py	403 Forbidden	1, pela rede
2	GET /%2e%2e/server.py (ponto codificado)	403 Forbidden	1, pela rede
3	GET /.%2fserver.py (barra codificada)	403 Forbidden	2, pela rede
4	GET /%2e%2e%2fserver.py (tudo codificado)	403 Forbidden	localhost
5	GET /%252e%252e/server.py (codificação dupla)	404 (vira um nome de arquivo que não existe)	localhost

Duas máquinas atendidas ao mesmo tempo

Na rodada 2, o A (192.168.1.12) e o B (192.168.1.13) pediram /a.txt ao mesmo tempo, 30 vezes cada. O log do servidor mostra os dois IPs misturados nos mesmos segundos (18:35:34 a 18:35:41), cada pedido em uma conexão diferente (portas de origem distintas), e as 60 respostas foram 200.

```
18:35:35 192.168.1.12:56230 "GET /a.txt" 200
18:35:36 192.168.1.12:56231 "GET /a.txt" 200
18:35:36 192.168.1.13:57613 "GET /a.txt" 200
18:35:36 192.168.1.12:56232 "GET /a.txt" 200
18:35:36 192.168.1.13:57614 "GET /a.txt" 200
18:35:36 192.168.1.12:56233 "GET /a.txt" 200
18:35:36 192.168.1.12:56234 "GET /a.txt" 200
18:35:36 192.168.1.13:57615 "GET /a.txt" 200
18:35:36 192.168.1.12:56235 "GET /a.txt" 200
18:35:36 192.168.1.12:56236 "GET /a.txt" 200
18:35:36 192.168.1.13:57616 "GET /a.txt" 200
18:35:36 192.168.1.12:56237 "GET /a.txt" 200
18:35:36 192.168.1.13:57617 "GET /a.txt" 200
18:35:36 192.168.1.12:56238 "GET /a.txt" 200
18:35:37 192.168.1.12:56239 "GET /a.txt" 200
```

Figura 1. Log do servidor (trecho): os IPs 192.168.1.12 e 192.168.1.13 aparecem misturados, todos com 200.

Conexão persistente e timeout

Teste	O que fizemos	O que vimos
Várias respostas na mesma conexão	curl com /a.txt e /b.txt na mesma linha	O log mostra os dois pedidos vindos da mesma porta de origem (61374), e o curl avisou que reaproveitou a conexão.
Fechar conexão parada	Conectar e não enviar nada	O servidor fechou a conexão depois de 5 s, sem enviar nada.

Captura de um pedido GET completo

No A, o Wireshark capturou um GET /a.txt feito pelo B com curl (rodada 2). A captura tem só 10 pacotes: 3 do handshake, 1 com o pedido, 2 com a resposta e 4 de encerramento. Os números de sequência e de confirmação são os relativos que o Wireshark mostra.

#	ms	Sentido	Flags	Dados (bytes)	Seq / Ack	O que é
1	0,0	B → A	SYN	0	seq 0	Handshake: o B pede para abrir a conexão.
2	0,2	A → B	SYN, ACK	0	seq 0, ack 1	Handshake: o A aceita.
3	2,1	B → A	ACK	0	seq 1, ack 1	Handshake pronto (do 2 ao 3 passou 1 RTT, uns 1,9 ms).
4	5,9	B → A	PSH, ACK	86	seq 1, ack 1	O pedido: GET /a.txt HTTP/1.1.
5	7,8	A → B	PSH, ACK	190	seq 1, ack 87	Resposta: 200 OK e os cabeçalhos.
6	8,1	A → B	PSH, ACK	14	seq 191, ack 87	Resposta: o corpo (conteúdo do arquivo).
7	17,8	B → A	FIN, ACK	0	seq 87, ack 205	Encerramento: o B fecha o seu lado.
8	17,9	A → B	ACK	0	seq 205, ack 88	Encerramento: o A confirma.
9	17,9	A → B	FIN, ACK	0	seq 205, ack 88	Encerramento: o A fecha o seu lado.
10	20,9	B → A	ACK	0	seq 88, ack 206	Encerramento: o B confirma e a conexão acaba.

Como interpretar: o ack 87 do pacote 5 é 1 + os 86 bytes do pedido, e o ack 205 do pacote 7 é 1 + 190 + 14 bytes da resposta. O FIN conta como 1 byte, por isso o ack 88 e o ack 206. A resposta saiu cerca de 2 ms depois do pedido.

Teste no navegador (interoperabilidade)

O navegador Chrome do B abriu <http://192.168.1.12:8080/> (rodada 2) e a página de teste do [www/](http://www.pysock.com/) apareceu completa: banner, logo, estilo e uma frase em verde que só aparece quando o script é executado. Para montar a página, o navegador pediu 5 arquivos (o HTML, o CSS, o banner, o logo e o script), todos com 200, num total de 26,5 kB e 232 ms (Figura 2). No log do servidor (Figura 3), o navegador usou conexões em paralelo e reaproveitou conexões: na segunda carga, a porta 55439 atendeu o HTML, o CSS e o script, e o banner e o logo vieram por outras duas conexões. O pedido do favicon.ico com 404 é o navegador pedindo o ícone da aba sozinho, e não atrapalha a página.

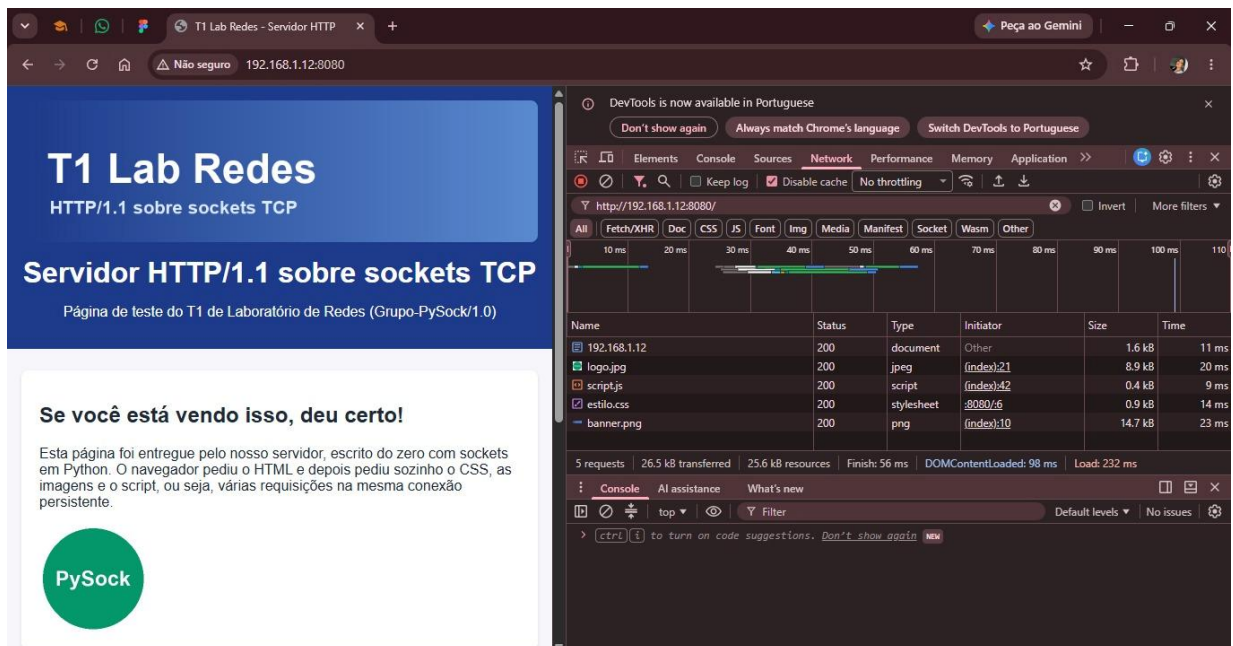


Figura 2. Página no Chrome do B, com a aba Network: 5 pedidos, todos 200.

```

22:04:02 192.168.1.13:53419 "GET /estilo.css" 200
22:04:02 192.168.1.13:61674 "GET /banner.png" 200
22:04:02 192.168.1.13:53339 "GET /logo.jpg" 200
22:04:02 192.168.1.13:53419 "GET /script.js" 200
22:04:02 192.168.1.13:61674 "GET /favicon.ico" 404
22:04:21 192.168.1.13:55439 "GET /" 200
22:04:21 192.168.1.13:55439 "GET /estilo.css" 200
22:04:21 192.168.1.13:56295 "GET /banner.png" 200
22:04:21 192.168.1.13:50110 "GET /logo.jpg" 200
22:04:21 192.168.1.13:55439 "GET /script.js" 200

```

Figura 3. Log do servidor no A com os pedidos do navegador (192.168.1.13).

4. CASOS DE TESTE

RTT entre os computadores (rodada 2)

O ping do B para o A (192.168.1.12), com 20 pacotes, teve 0% de perda: mínimo de 3 ms, média de 5 ms e máximo de 13 ms, então usamos a média de 5 ms como RTT. O ping arredonda para milissegundos inteiros, então o handshake da própria captura serve de conferência: ele ficou entre 1,8 ms e 3,4 ms.

```
PS C:\WINDOWS\system32> ping -n 20 192.168.1.12

Disparando 192.168.1.12 com 32 bytes de dados:
Resposta de 192.168.1.12: bytes=32 tempo=3ms TTL=128
Resposta de 192.168.1.12: bytes=32 tempo=5ms TTL=128
Resposta de 192.168.1.12: bytes=32 tempo=5ms TTL=128
Resposta de 192.168.1.12: bytes=32 tempo=13ms TTL=128
Resposta de 192.168.1.12: bytes=32 tempo=4ms TTL=128
Resposta de 192.168.1.12: bytes=32 tempo=9ms TTL=128
Resposta de 192.168.1.12: bytes=32 tempo=10ms TTL=128
Resposta de 192.168.1.12: bytes=32 tempo=5ms TTL=128
Resposta de 192.168.1.12: bytes=32 tempo=11ms TTL=128
Resposta de 192.168.1.12: bytes=32 tempo=6ms TTL=128
Resposta de 192.168.1.12: bytes=32 tempo=4ms TTL=128
Resposta de 192.168.1.12: bytes=32 tempo=4ms TTL=128
Resposta de 192.168.1.12: bytes=32 tempo=5ms TTL=128
Resposta de 192.168.1.12: bytes=32 tempo=5ms TTL=128
Resposta de 192.168.1.12: bytes=32 tempo=7ms TTL=128
Resposta de 192.168.1.12: bytes=32 tempo=4ms TTL=128
Resposta de 192.168.1.12: bytes=32 tempo=4ms TTL=128
Resposta de 192.168.1.12: bytes=32 tempo=4ms TTL=128
Resposta de 192.168.1.12: bytes=32 tempo=4ms TTL=128
Resposta de 192.168.1.12: bytes=32 tempo=5ms TTL=128
Resposta de 192.168.1.12: bytes=32 tempo=4ms TTL=128

Estatísticas do Ping para 192.168.1.12:
    Pacotes: Enviados = 20, Recebidos = 20, Perdidos = 0 (0% de
    perda),
    Aproximar um número redondo de vezes em milissegundos:
    Mínimo = 3ms, Máximo = 13ms, Média = 5ms
```

Figura 4. Ping do B para o A: 20 respostas, média de 5 ms.

Comparação entre C1 e C2

O medir.py, no B, fez 10 pedidos seguidos a /a.txt. Em C1, cada pedido abriu uma conexão nova (Connection: close). Em C2, os 10 pedidos usaram uma conexão só. Cada cenário foi capturado separadamente com o Wireshark, no A, e os números saíram da captura (capturas/c1.pcapng e capturas/c2.pcapng).

Métrica	C1 (10 conexões)	C2 (1 conexão)	Economia de C2
Handshakes TCP completos	10	1	não se aplica
Total de pacotes	108	40	63,0%
Bytes totais	8452	4861	42,5%
Tempo total (primeiro ao último pacote)	121,0 ms	70,9 ms	41,4%

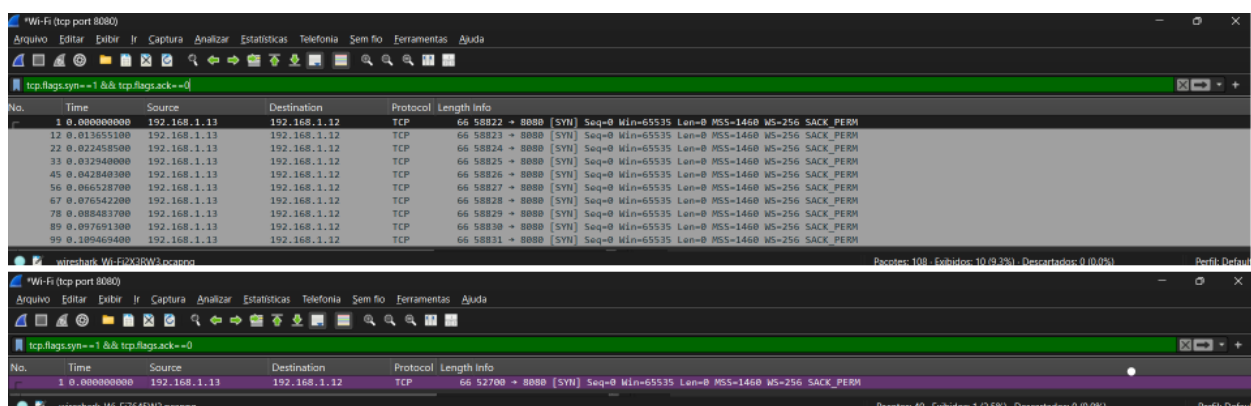


Figura 5. Wireshark no A, com o filtro de SYN: C1 (em cima) tem 10 conexões e C2 (embaixo) tem 1.

O medir.py também mostra o tempo, mas no B ele marcou cerca de 100 ms a mais que a captura nos dois cenários (225,1 ms e 166,7 ms). Por isso usamos o tempo da captura, que só conta o que passou pela rede.

Custo de abrir e fechar conexões em C1

Cada conexão de C1 gasta 3 pacotes só para abrir (o handshake) e mais alguns para fechar. Na captura, abrir e fechar as 10 conexões somou 78 pacotes e 4452 bytes, ou seja, 72% dos pacotes e 53% dos bytes de C1 (30 pacotes e 1860 bytes só de handshake). Em C2, abrir e fechar a única conexão custou 7 pacotes e 402 bytes.

Quantos RTTs a mais o C1 gasta

Em C1, cada pedido paga 1 RTT para o handshake e mais 1 RTT para o pedido e a resposta, cerca de 20 RTT no total. Em C2, o handshake é pago uma vez só: 1 + 10 = 11 RTT. A diferença esperada é de 9 RTT, um para o handshake de cada conexão nova depois da primeira.

Medimos $T_{C1} = 121,0$ ms e $T_{C2} = 70,9$ ms, uma diferença de 50,1 ms. Dividindo pelo RTT de 5 ms, o C1 gastou **10,0 RTT** a mais, bem perto dos 9 previstos. Os 50,1 ms divididos pelas 9 conexões extras dão 5,6 ms por conexão, um pouco mais que o handshake sozinho (uns 2 ms). Nossa hipótese é que cada conexão nova também gasta tempo para ser criada e fechada, mas a captura sozinha não prova isso.

5. CONSIDERAÇÕES FINAIS

O servidor funcionou como esperado nos testes entre dois computadores: respondeu 200, 400, 403, 404 e 405, barrou as tentativas de sair da pasta raiz, manteve a conexão aberta para vários pedidos, fechou conexões paradas em 5 s e atendeu duas máquinas ao mesmo tempo. Também testamos a página de teste do `www/` no navegador de outro computador, e os 5 arquivos (HTML, CSS, duas imagens e um script) carregaram com 200 (Figuras 2 e 3).

Quando a conexão persistente ganha mais

Cada conexão evitada economiza um handshake, e cada handshake custa cerca de 1 RTT. Então o ganho de tempo é de mais ou menos (número de conexões evitadas) $\times$ RTT: quanto mais conexões e quanto maior o RTT, maior o ganho. Na nossa rede local, com RTT de 5 ms, a conexão persistente economizou 63,0% dos pacotes, 42,5% dos bytes e 41,4% do tempo (uns 50 ms). Em uma rede com RTT alto (um enlace longo, Wi-Fi ruim, rede de celular ou satélite) e com páginas de muitos arquivos, o ganho em tempo seria bem maior.

O que deu errado no caminho e o que aprendemos:

- **O ping perdeu tudo, mas o servidor respondia:** O firewall do A bloqueava o ping. Nossa primeira regra valia só para redes "Private" e o Wi-Fi estava como "Public". Criamos uma regra para Public, só para a rede local, e depois removemos as regras.
- **A conexão persistente saiu mais lenta que a normal:** Na primeira medição, C2 levou 543,8 ms e C1 só 91,2 ms. Na captura vimos que o servidor mandava o cabeçalho e o corpo da resposta separados, e o corpo esperava uns 50 ms por uma confirmação do cliente, uma regra do TCP para economizar pacotes pequenos. Ligamos o `TCP_NODELAY` no servidor (uma linha no código) e refizemos as capturas. Os números do relatório são da medição corrigida.
- **Pequenos tropeços:** PowerShell em system32 (não cria arquivos) e um 404 em `/b.txt` (o arquivo ainda não existia).
- **Limitações:** trabalho de estudo, só com GET e HEAD, sem HTTPS, uma thread por conexão, testado somente em redes locais.